
Grove Basin Audit Report

Prepared by Riley Holterhus
April 8, 2026

Contents

1	Introduction	3
1.1	About Grove Basin	3
1.2	About the Auditor	3
1.3	Disclaimer	3
2	Audit Overview	4
2.1	Scope of Work	4
2.2	Summary of Findings	4
3	Findings	5
3.1	Medium Severity Findings	5
3.1.1	totalAssets() is undervalued during redemptions	5
3.1.2	UsdsUsdcPocket dual-token design is not fully accounted for	5
3.2	Low Severity Findings	6
3.2.1	Incorrect interface used for USDC/USDS conversions in UsdsUsdcPocket	6
3.2.2	Nested ceilDiv() calls in swap conversions amplify rounding errors	6
3.2.3	roundUp parameter is ignored for swap and collateral token valuations	7
3.2.4	Some calculations assume rate provider precision without checking	7
3.2.5	Hardcoded 1e6 initial deposit is not decimal-aware	8
3.2.6	removeTokenRedeemer() does not check for ongoing redemptions	8
3.3	Informational Findings	9
3.3.1	No slippage protection on deposit() or withdraw()	9
3.3.2	Deploy.s.sol targets Base instead of mainnet	9
3.3.3	GroveBasinDeploy library is not necessary	9
3.3.4	Swap tokens not always automatically deposited into yield strategy	10
3.3.5	aToken approval to Aave pool is unnecessary for withdrawals	10
3.3.6	type(uint256).max sentinel value reachable in AaveV3UsdtPocket	10
3.3.7	Pocket withdrawals can be temporarily unavailable	11
3.3.8	availableBalance() must never overestimate deliverable swapToken	11
3.3.9	maxSwapSize comparisons use a rounded-down asset value	12
3.3.10	Fee rounding in previewSwapExactIn() favors the user	12
3.3.11	Unnecessary creditToken approval in JTRSYTokenRedeemer	12

1 Introduction

1.1 About Grove Basin

Grove Basin is a multi-asset liquidity pool built as a fork of Spark's PSM3 contract. It holds three token types: a credit token, a collateral token, and a swap token. The contract issues shares representing a proportional claim on the pool's total value. Depositing and withdrawing shares is permissioned. Swapping between the three token types is the only permissionless action in the contract. Swaps must always be to or from the credit token, and exchange rates are determined by external oracle rate providers. A fee is charged on each swap. Deposited swap tokens may be deployed into external yield strategies through a "pocket" system, which currently supports AaveV3, Morpho Vaults, and MakerDAO's Peg Stability Module (called the UsdsUsdcPocket). The contract also supports permissioned credit token redemptions through a redeemer system. Currently, a jTRSY redeemer contract is included for converting jTRSY credit tokens back into USDC. For more information about Grove, visit their website: grove.finance.

1.2 About the Auditor

Riley Holterhus is an independent smart contract security researcher that focuses on Solidity smart contracts. Other than conducting independent security reviews, he works as a Lead Security Researcher at [Spearbit](https://spearbit.com), and also searches for vulnerabilities in live codebases. Riley can be reached by email at rileyholterhus@gmail.com, by Telegram at [@holterhus](https://t.me/holterhus) and on Twitter/X at [@rileyholterhus](https://twitter.com/rileyholterhus).

1.3 Disclaimer

This report is intended to detail the identified vulnerabilities of the reviewed smart contracts and should not be construed as an endorsement or recommendation of any project, individual or entity. While the authors have made reasonable efforts to detect potential issues, the absence of any undetected vulnerabilities or issues cannot be guaranteed. Additionally, the security of the smart contracts may be affected by future changes or updates. By using the information in this report, you acknowledge that you are doing so at your own risk and that you should exercise your own judgment when implementing any recommendations or making decisions based on the findings. This report has been provided on an "as-is" basis and DOES NOT CONSTITUTE A GUARANTEE OR WARRANTY OF ANY FORM.

2 Audit Overview

2.1 Scope of Work

From March 24th, 2026 through March 27th, 2026, Riley Holterhus conducted an audit of Grove's Basin smart contracts. During this period, a manual analysis was undertaken to identify various security issues and logic flaws. The audit was performed on the codebase found in the [grove-labs/grove-basin](#) GitHub repository. The audit started on commit [da15116f69594842a35c7fc303bd24f15b6895a8](#). The files in scope for the audit were:

```
src/  
  GroveBasin.sol  
  GroveBasinFactory.sol  
  JTRSYTokenRedeemer.sol  
  interfaces/  
    (all files)  
  pockets/  
    AaveV3UsdtPocket.sol  
    BasePocket.sol  
    MorphoUsdtPocket.sol  
    UsdsUsdcPocket.sol  
  rate-providers/  
    ChronicleRateProvider.sol  
    FixedRateProvider.sol  
deploy/  
  GroveBasinDeploy.sol  
script/  
  Deploy.s.sol  
  DeployChronicleRateProvider.s.sol  
  DeployFixedRateProvider.s.sol  
  SetupJTRSYMorphoUsdtBasin.s.sol  
  SetupJTRSYUsdsUsdcBasin.s.sol
```

2.2 Summary of Findings

Each finding from the audit has been assigned a severity level of “Critical”, “High”, “Medium”, “Low” or “Informational”. These severities aim to capture the impact and likelihood of each potential issue.

In total, **19 findings** were identified: 2 Medium, 6 Low, and 11 Informational. The description and status of the findings are provided in the sections below.

Note that during the mitigation review, fixes for findings were reviewed. However, some commits and PRs referenced in the finding statuses also included additional changes outside the scope of those specific fixes, and those broader changes were not reviewed as part of this process. As a result, references to mitigation commits or PRs should not be interpreted as a full review of all changes they contain. The team is undergoing a further external audit following this review.

3 Findings

3.1 Medium Severity Findings

3.1.1 `totalAssets()` is undervalued during redemptions

Description: The `totalAssets()` function excludes credit tokens that are pending redemption, even though the `GroveBasin` still has a claim on their value. This deflates the share price during the redemption period. A depositor who enters during this window gets shares at a lower price and profits when `completeRedeem()` brings the swap tokens back, extracting value from existing LPs. If all depositors and withdrawers are the same entity, the issue doesn't apply, but it does if there are multiple independent LPs.

Recommendation: The simplest fix would be to use `totalAssetsWithRedemptions()` for share pricing, which values pending credit tokens at their current rate. This isn't perfectly accurate because the actual redemption exchange rate may have been locked in earlier, and the credit token continues accruing value past that point.

Another potential approach would be to snapshot the asset value at the time of the redemption request, but this is harder to implement: there can be multiple redeemers, the `GroveBasin` would need to track pending asset values separately, and some redeemers (like the `JTRSYTokenRedeemer`) don't lock in the exchange rate at the moment of the request anyway.

Regardless, valuing the pending tokens closer to the current credit rate is much closer to correct than valuing them at zero. If the true value is x , the current behavior has an error of x , while using the current rate has an error of $|x' - x|$. For slowly-accruing credit tokens, $|x' - x|$ will be negligible compared to x . Of these options, changing to the `totalAssetsWithRedemptions()` behavior seems like the best tradeoff between simplicity and accuracy.

Grove: Addressed in [PR 45](#).

Auditor: Verified. The `pendingCreditTokenBalance` is now added to the credit token amount passed to `_getCreditTokenValue()` within `totalAssets()`. This means that credit tokens that are pending redemption will be valued the same as regular credit tokens.

3.1.2 `UsdsUsdcPocket` dual-token design is not fully accounted for

Description: The `UsdsUsdcPocket` is unique among the pocket implementations because it holds both the swap token (USDS) and the collateral token (USDC). This dual-token behavior causes two separate issues:

- The `setPocket()` function only migrates the swap token from the old pocket to the new one. Any collateral token sitting in the old pocket is left behind.
- The `_getAvailableBalance(collateralToken)` call only counts collateral held directly by the `GroveBasin`, not inside the pocket. This means the collateral token in the pocket is excluded from `totalAssets()`, which affects share pricing. This is partly because `availableBalance(USDS)` does not include the pocket's USDC balance, even though `availableBalance(USDC)` does include the USDS balance (but since USDC is the collateral token, the `availableBalance(USDC)` path is never reached anyway). Another effect is that the collateral token in the pocket cannot be used for swaps or other operations.

Recommendation: Consider whether the `UsdsUsdcPocket` needs to hold the collateral token directly, or if the design can be restructured to avoid the dual-token behavior. It may also be worth reconsidering which token is designated as the swap token versus the collateral token in this configuration. If holding both tokens in the pocket is necessary, the `GroveBasin` code should be updated to handle this more generically, particularly in `setPocket()` and `_getAvailableBalance()`.

Grove: Acknowledged.

Auditor: Acknowledged.

3.2 Low Severity Findings

3.2.1 Incorrect interface used for USDC/USDS conversions in `UsdsUsdcPocket`

Description: The `UsdsUsdcPocket` converts between USDC and USDS using `swapExactIn()` and `swapExactOut()`, which are functions in the PSM3 interface. However, the PSM3 contracts are not currently deployed on Ethereum mainnet, which is where the `GroveBasin` code will be deployed. Because of this, the implementation should instead use the interface of the deployed mainnet contract `0xa188eec8f81263234da3622a406892f3d630f98c`, which uses `sellGem()` and `buyGem()` for conversions between USDC and USDS. This is the main contract used for USDS/USDC conversions on mainnet.

Recommendation: Change the `UsdsUsdcPocket` to use `buyGem()` and `sellGem()` for USDS/USDC conversions.

Grove: Fixed in [commit ddcdbae](#).

Auditor: Verified. Note that the mainnet contract at `0xa188eec8f81263234da3622a406892f3d630f98c` currently has both `tout` and `tin` fee values set to 0. If either were non-zero, this would affect whether the USDS/USDC conversions assumed in `availableBalance()` are accurate.

3.2.2 Nested `ceilDiv()` calls in swap conversions amplify rounding errors

Description: The swap conversion functions use two nested `ceilDiv()` calls in their round-up branches, which can amplify rounding errors.

For example, `_convertSwapToCreditToken()` computes:

```
return Math.ceilDiv(
    Math.ceilDiv(amount * swapRate, creditRate) * _creditTokenPrecision,
    _swapTokenPrecision
);
```

With a 6-decimal swap token, 18-decimal credit token, `amount = 1`, `swapRate = 1e27`, `creditRate = 3e27`, this evaluates to:

```
ceilDiv(ceilDiv(1 * 1e27, 3e27) * 1e18, 1e6)
= ceilDiv(1 * 1e18, 1e6)
= 1e12
```

The first `ceilDiv` rounds $0.333\dots$ up to 1, and that error is multiplied by $1e18 / 1e6 = 1e12$ in the second step. A single `ceilDiv` over the full expression would give 333333333334 (error of +1). The nested version gives 1000000000000 (error of +666666666666).

Recommendation: Consider replacing the nested `ceilDiv()` with a single division to avoid error amplification. Because overflow in the intermediate product may be a concern, consider using `Math.mulDiv` with `Rounding.Ceil`, which handles the multiplication in 512 bits internally.

Grove: Fixed in [commit 4530ffa](#).

Auditor: Verified.

3.2.3 `roundUp` parameter is ignored for swap and collateral token valuations

Description: In `_getAssetValue()`, the `roundUp` parameter is only passed through to the credit token path. The `_getSwapTokenValue()` and `_getCollateralTokenValue()` functions always round down regardless of what `roundUp` is set to.

The only place in the code that passes `roundUp = true` is `previewWithdraw()`, where the intent is to round up the asset value before passing it to `_convertToSharesRoundUp()` to determine the number of shares to burn from the user. Because the rounding is not applied for swap and collateral tokens, the number of shares burned can favor the user over the protocol.

In the worst case, the value can round down to zero. For example, with an 18-decimal token, if the rate provider returns \$0.50 (i.e. $5e26$), then `_getSwapTokenValue(1)` evaluates to $(1 * 5e26 * 1e18) / (1e27 * 1e18)$ which rounds to 0. The user would withdraw 1 wei while burning zero shares.

The amounts involved are likely small, but the possibility of a withdrawal that burns zero shares is still an invariant worth fixing.

Recommendation: Add rounding support to `_getSwapTokenValue()` and `_getCollateralTokenValue()` the same way `_getCreditTokenValue()` already has it.

Grove: Fixed as part of [PR 47](#), and added related testing in [commit f19604f](#).

Auditor: Verified. The `_getSwapTokenValue()` and `_getCollateralTokenValue()` functions now accept a `bool roundUp` value and pass it through to the `mulDiv()` call.

3.2.4 Some calculations assume rate provider precision without checking

Description: Some calculations in the `GroveBasin` use values returned by rate providers without incorporating the provider's precision. This works correctly with the `FixedRateProvider` and `ChronicleRateProvider` since they

return rates at the expected 1e27 precision, but the code is fragile if a new provider with a different precision is introduced. The places where the precision is not accounted for are:

- The four swap conversion functions (`_convertSwapToCreditToken()`, `_convertCollateralToCreditToken()`, `_convertCreditTokenToSwap()`, `_convertCreditTokenToCollateral()`) each use rates from two different providers in the same formula without accounting for their precision. If the two providers return rates at different precisions, the rates do not cancel cleanly and the result is incorrect.
- The `convertToAssets()` function hardcodes 1e9 in its formula, which comes from $1e27 / 1e18$, where 1e18 is the USD value precision used in the code. A provider returning rates at a precision other than 1e27 would make `convertToAssets()` incorrect.

Recommendation: Incorporate `getRatePrecision()` into the conversion functions and `convertToAssets()`.

Grove: Fixed in [commit 9250aec](#).

Auditor: Verified. For intermediate overflow concerns, it's worth considering whether expressions such as `creditRate * _collateralTokenPrecision * collateralRatePrecision` could overflow before being passed to `mulDiv()`. For example, $1e27 * 1e18 * 1e27 = 1e72$ is below `type(uint256).max`, but higher provider precisions or large rate values could reach overflows. It may be worth refactoring the formula to reduce intermediate values, for instance, by cancelling precision terms earlier. One example is to compare the different provider precision first and only apply the relevant difference into the calculation.

3.2.5 Hardcoded 1e6 initial deposit is not decimal-aware

Description: The `GroveBasinFactory` deposits 1e6 of the `swapToken` as a seed deposit to prevent the first-deposit share inflation attack. This amount is hardcoded regardless of the token's decimals. For tokens with a smaller number of decimal places, this results in a large deposit. For example, [GUSD](#) has 2 decimal places, so 1e6 raw units is \$10,000 permanently locked as dead shares.

Recommendation: Consider making the seed deposit amount a configurable parameter in the `deploy()` function, with a non-zero minimum to ensure the first-deposit attack prevention is still in place.

Grove: Fixed in [commit 13b4585](#).

Auditor: Verified. The new logic now deposits $10^{** \text{IERC20}(\text{swapToken}).\text{decimals}()}$ of the `swapToken` as the initial amount. This fixes the issue mentioned with GUSD, since a deposit of \$1 would be made instead of \$10,000. Note that for a non-stablecoin `swapToken`, one full unit could still be a large amount depending on the token's value.

3.2.6 `removeTokenRedeemer()` does not check for ongoing redemptions

Description: There is nothing preventing a redeemer with an ongoing redemption from being removed via `removeTokenRedeemer()`. If this happens, the in-flight redemption would be lost unless the redeemer is re-added, since only the `Basin` can call the `completeRedeem()` function.

Note that a straightforward on-chain check would not be easy to implement, because `redeemedCreditTokenBalance` is a global value shared across all redeemers, not tracked per redeemer.

Recommendation: Admins should verify that there are no ongoing redemptions before calling `removeTokenRedeemer()`. As an additional safeguard, consider temporarily setting `initiateRedeemPaused = true` when adding or removing redeemers, so that new redemptions cannot frontrun the redeemer removal call.

Grove: Addressed as part of the changes in [PR 55](#).

Auditor: Verified. The redemption logic now tracks the number of pending redemptions for each redeemer. Note that there are other changes in PR 55 that were not in scope of the mitigation review.

3.3 Informational Findings

3.3.1 No slippage protection on `deposit()` or `withdraw()`

Description: The `deposit()` and `withdraw()` functions don't take slippage parameters, so callers can't commit to a specific minimum or maximum exchange rate. If the share price or fee values change between submitting a transaction and when it is actually executed, the caller has no way to enforce their expected rate.

Recommendation: Consider adding slippage parameters to `deposit()` and `withdraw()`.

Grove: Acknowledged, will not fix; we'd prefer to keep the PSM3 ABI here, and a single trusted LP mitigates the risk around share/fee values changing. Will document for the LP.

Auditor: Acknowledged.

3.3.2 `Deploy.s.sol` targets Base instead of mainnet

Description: The `DeployEthereum` contract in `script/Deploy.s.sol` uses `getChain("base").rpcUrl` to create its fork. This should be `getChain("mainnet").rpcUrl`. The script does not appear to be used anywhere and has unset parameters, so it may be okay to delete.

Recommendation: Consider deleting the `script/Deploy.s.sol` file.

Grove: Fixed in [commit fee9532](#).

Auditor: Verified.

3.3.3 `GroveBasinDeploy` library is not necessary

Description: The `GroveBasinDeploy` library and the `GroveBasinFactory` contract implement the same deployment logic. The deploy scripts currently use the library rather than the factory. Beyond the code duplication concern, using the library from a deploy script means the deployer EOA is `msg.sender` during deployment, so the EOA receives the `LIQUIDITY_PROVIDER_ROLE`. This is likely not the intended long-term role assignment.

Recommendation: Consider deleting the `GroveBasinDeploy` library and update the deploy scripts to use `GroveBasinFactory` directly.

Grove: Fixed in [commit 999c02c](#).

Auditor: Verified. The `GroveBasinDeploy` library has been removed, and the `GroveBasinFactory` is now being used. Note that both `SetupJTRSYMorphoUsdtBasin` and `SetupJTRSYUsdsUsdcBasin` deploy a new factory within the script. Consider refactoring this in the future so they use a single shared factory instead, potentially once the factory itself is deployed.

3.3.4 Swap tokens not always automatically deposited into yield strategy

Description: In `swapExactIn()` and `swapExactOut()`, when `assetIn == swapToken`, the tokens are transferred to the pocket but `_depositLiquidityInPocket()` is never called. The same is true for `setPocket()`, which migrates swap tokens from the old pocket to the new one without triggering a deposit into the new pocket's yield strategy. In both cases, swap tokens sit idle in the pocket until a separate action deploys them.

In contrast, the `deposit()` function always calls `_depositLiquidityInPocket()` after transferring the swap token into the pocket.

Recommendation: Consider calling `_depositLiquidityInPocket()` after transferring swap tokens in these paths, or document this as intentional behavior that relies on periodic manual deposits by managers.

Grove: Acknowledged. We'll document this as intentional behavior that relies on managers.

Auditor: Acknowledged.

3.3.5 aToken approval to Aave pool is unnecessary for withdrawals

Description: The `AaveV3UsdtPocket` approves the `aToken` to the Aave pool before calling `pool.withdraw()`. This approval is unnecessary. The pool calls `burn()` on the `aToken` directly using its own burning privileges, and never checks or consumes any ERC20 allowance.

Recommendation: Remove the `aToken` approval from `withdrawLiquidity()` in the `AaveV3UsdtPocket`.

Grove: Fixed in [commit a275892](#).

Auditor: Verified.

3.3.6 `type(uint256).max` sentinel value reachable in `AaveV3UsdtPocket`

Description: In Aave V3, `pool.withdraw()` treats `amount == type(uint256).max` as a special sentinel value meaning "withdraw my full balance."

This value is not reachable through normal `GroveBasin` operations, since the `GroveBasin` caps withdrawal amounts based on available balances. However, a manager could directly call `withdrawLiquidity(type(uint256).max)`,

USDT) on the AaveV3UsdtPocket. If the pocket's direct USDT balance happens to be zero, this results in passing `type(uint256).max - 0` to Aave's `withdraw()` function.

The net effect is the same as withdrawing the full balance normally, but the emitted event and return value would show `type(uint256).max` rather than the actual amount withdrawn, which could be misleading.

Since managers are trusted and the outcome is functionally identical to a full withdrawal, this is a minor concern.

Recommendation: Consider explicitly disallowing `amount == type(uint256).max` in the AaveV3UsdtPocket.

Grove: Fixed in [commit 5b85a88](#).

Auditor: Verified.

3.3.7 Pocket withdrawals can be temporarily unavailable

Description: The pocket contracts depend on external protocols (Aave, Morpho), and those protocols may not always have enough liquidity to honor a withdrawal. For example, if the Aave pool has high utilization, `withdrawLiquidity()` will revert because the pool cannot fulfill the request. The other pockets can reach similar conditions depending on their underlying protocol. This does not result in any permanent loss of funds, but it can temporarily delay operations that need to pull tokens from the pocket (e.g. `setPocket()`).

Recommendation: Keep this behavior in mind.

Grove: Acknowledged. We will document and rely on the trusted manager to monitor lending pools for liquidity crunches.

Auditor: Acknowledged.

3.3.8 `availableBalance()` must never overestimate deliverable `swapToken`

Description: Several parts of the GroveBasin code call `pocket.availableBalance(swapToken)` and will potentially attempt to withdraw up to that amount from the pocket. If `availableBalance()` ever overestimates what the pocket can actually deliver, even by 1 wei, the subsequent withdrawal or transfers could revert. This could happen due to share-to-asset rounding in the underlying protocol, fees that the `availableBalance()` doesn't consider, or any other discrepancy between the reported balance and the final realizable amount.

All of the current pocket implementations seem correct and do not overestimate. So this finding has been provided as an informational note.

Recommendation: Keep in mind this key invariant for any future pocket implementation: `availableBalance()` must never report more `swapToken` than the pocket can actually deliver. Implementations should always account for rounding, fees that may apply between the read and the withdrawal, and similar discrepancies.

Grove: Acknowledged.

Auditor: Acknowledged.

3.3.9 maxSwapSize comparisons use a rounded-down asset value

Description: The maxSwapSize checks use `roundUp = false` in `_getAssetValue()`, which means the value compared maxSwapSize is rounded down. This gives the user more leeway on the check. Since asset values are in 18-decimal USD precision, this is very negligible difference.

Recommendation: No changes appear necessary, keep this behavior in mind.

Grove: Acknowledged.

Auditor: Acknowledged.

3.3.10 Fee rounding in `previewSwapExactIn()` favors the user

Description: In `previewSwapExactIn()`, the fee is calculated with `roundUp = false` and subtracted from `amountOut`. Rounding the fee down produces a smaller subtraction, which gives the user a larger output. In `previewSwapExactOut()`, the fee is rounded up and added to `amountIn`, which rounds against the user. The two functions are asymmetric in their rounding direction.

Recommendation: Consider rounding the fee up in `previewSwapExactIn()` as well to consistently round against the user in both swap directions.

Grove: Fixed within [PR 47](#).

Auditor: Verified. The new logic is now always rounding up the fee amount charged to the user.

3.3.11 Unnecessary `creditToken` approval in `JTRSYTokenRedeemer`

Description: The `initiateRedeem()` function calls `IERC20(creditToken).approve(vault, creditTokenAmount)` before calling `vault.requestRedeem()`. This approval is never consumed by the `vault` code, and can be removed.

Recommendation: Remove the `creditToken` approval from `initiateRedeem()`.

Grove: Fixed in [commit 69418de](#).

Auditor: Verified.